Newton School of Technology

4

Polymorphism & Abstraction

By: Aditya Kumar

Newton School of Technology

Join the lecture online on your dashboard.

Recap of Previous Lecture!

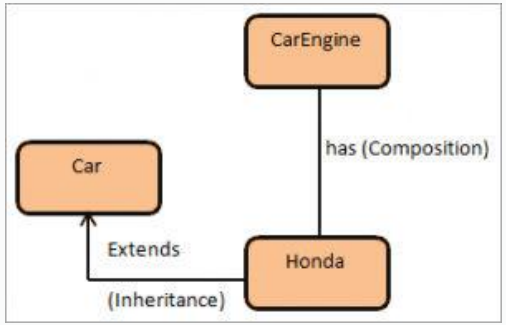
Quick Recap

What is OOP?

Object Oriented Programming helps us organize code into objects that work together, like building blocks that fit perfectly.

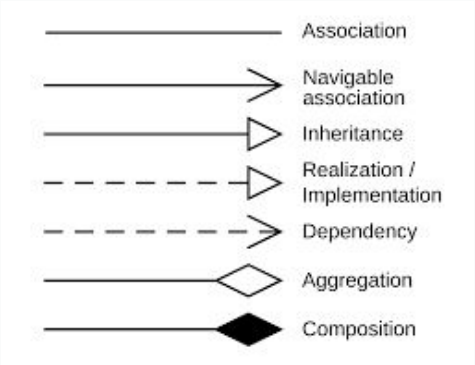
Inheritance & Composition

We learned how classes can be families - children inherit from parents. But sometimes it's better to use pieces instead of forcing relationships.



UML Arrows

Simple diagrams show family relationships between classes, making it easy to see who's the parent and who's the child.

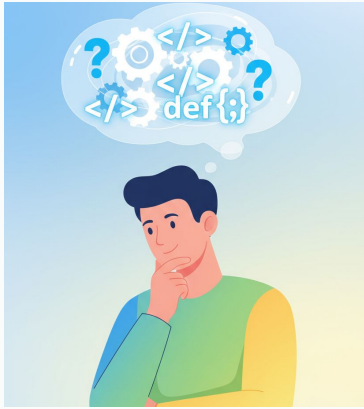


Polymorphism and Abstraction in OOP

Today's Big Questions?

1. **How can one thing do many different jobs?** → That's Polymorphism! (Think: Coffee Machine)

2. **How can we use something without knowing how it works?** → That's Abstraction! (Think: Driving a Car)



3. **How do these make us better programmers?** → We'll find out together!

What is Polymorphism?

- **Poly** = Many, **Morphism** = Forms
- Definition: The ability of objects to take multiple forms
- One interface, multiple implementations
- Key benefit: Write flexible and reusable code

Real-Life Example - The Coffee Machine

Real-Life Example - The Coffee Machine

Think of a coffee machine with one button labeled "BREW"

- Press BREW → Select Espresso → Get Espresso
- Press BREW → Select Cappuccino → Get Cappuccino
- Press BREW → Select Latte → Get Latte

Same button (interface), different outputs (behavior)

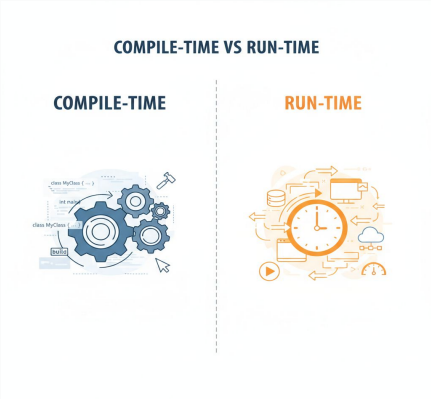
Types of Polymorphism

1. Compile-Time Polymorphism (Method Overloading)

- Same method name, different parameters
- Decided during compilation

2. Runtime Polymorphism (Method Overriding)

- Parent-child class relationship
- Decided during program execution



Types of Polymorphism

1. SubType Polymorphism

- Dog is a subtype of Animal, so it can be used wherever an Animal is expected.

2. Interface Based Polymorphism

- You write code against an interface (contract), and any class (or object) that implements that contract can be used. This is still subtype polymorphism—the interface is the supertype.

Example - Method Overloading

```
class Calculator {
  add(a: number, b: number): number;
  add(a: number, b: number, c: number): number;
  add(a: string, b: string): string;

  add(a: number | string, b: number | string, c?: number): number | string {
    if (typeof a === "number" && typeof b === "number") {
      if (typeof c === "number") return a + b + c;
      return a + b;
    }

    return String(a) + String(b);
  }
}
```

Example - Method Overriding

```
class Animal {
  speak() {
    console.log("Some sound");
  }
}
class Dog extends Animal {
  speak() {
    console.log("Bark");
  }
}
class Cat extends Animal {
  speak() {
    console.log("Meow");
  }
}
```

Benefits of Polymorphism

- **Code Reusability** - Write once, use many times
- **Flexibility** - Easy to add new features
- **Maintainability** - Changes in one place reflect everywhere
- **Cleaner Code** - Less redundancy, more organized

Task(Method Overloading)

Problem: Create a class called Printer that can print different types of data.

Your Task: Create three overloaded methods named print() that can:

1. Print a string message
2. Print an integer number
3. Print an integer number multiple times (takes number and count)

Task(Method Overriding)

Problem: Create a simple payment system for different payment methods.

Your Task:

1. Create a parent class Payment with a method processPayment()
2. Create two child classes: CreditCard and UPI
3. Each child class should override processPayment() with its own message

What is Abstraction?

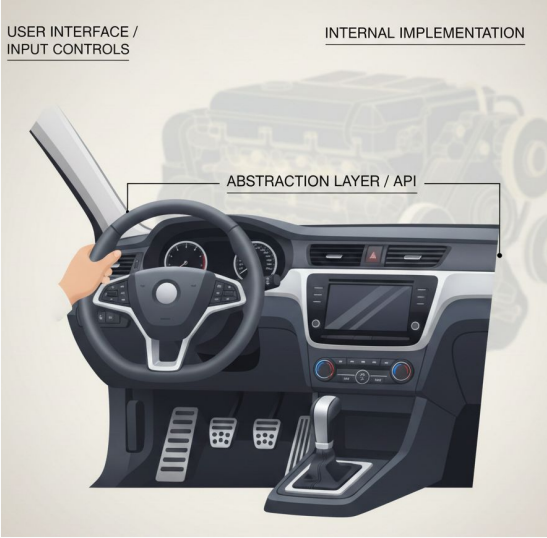
- Definition: Hiding complex implementation details and showing only essential features
- Focus on "WHAT it does" not "HOW it does"
- Example:- Like using a TV remote - you don't need to know the electronics inside
- Achieved through Abstract Classes and Interfaces

Real-Life Example - The Car

When you drive a car:

- **What you see:** Steering wheel, pedals, gear shift
- **What you don't see:** Engine mechanics, fuel injection system, transmission details

You interact with simple controls (abstraction) without worrying about complex internal mechanisms.



Abstract Class vs Interface

Abstract Class:

- Can have both abstract and concrete methods
- Can have variables
- Use: When classes share common behavior
- Keyword: abstract

Interface:

- Only abstract methods
- Cannot have variables
- Use: When classes need to implement a contract
- Keyword: interface

Example - Abstract Class

```
abstract class Vehicle {
    // Abstract method (no body)
    abstract start(): void;

    // Concrete method (has body)
    fuel(): void {
        console.log("Vehicle needs fuel");
    }
}

class Car extends Vehicle {
    start(): void {
        console.log("Car starts with key");
    }
}

class Bike extends Vehicle {
    start(): void {
        console.log("Bike starts with kick");
    }
}

// Usage
const myCar = new Car();
myCar.start();  // Output: Car starts with key
myCar.fuel();   // Output: Vehicle needs fuel

const myBike = new Bike();
myBike.start(); // Output: Bike starts with kick
```

Vehicle is abstract - you can't create a generic vehicle, but you can create specific types like Car or Bike.

Example - Interface

```
interface Playable {
    play(): void;
    pause(): void;
    stop(): void;
}

class MusicPlayer implements Playable {
    play(): void {
        console.log("Playing music...");
    }

    pause(): void {
        console.log("Music paused");
    }

    stop(): void {
        console.log("Music stopped");
    }
}

class VideoPlayer implements Playable {
    play(): void {
        console.log("Playing video...");
    }

    pause(): void {
        console.log("Video paused");
    }

    stop(): void {
        console.log("Video stopped");
    }
}

// Usage
const music: Playable = new MusicPlayer();
const video: Playable = new VideoPlayer();

music.play();    // Output: Playing music...
video.play();    // Output: Playing video...
```

Both players implement the same contract (Playable interface) but in their own way.

Benefits of Abstraction

- **Reduces Complexity** - Hide unnecessary details
- **Increases Security** - Expose only what's needed
- **Enhances Maintainability** - Changes don't affect users
- **Promotes Modularity** - Break code into manageable pieces

Coding Activity (Abstraction)

Create a simple shape calculator using abstraction.

Your Task:

1. Create an abstract class Shape with:
   - An abstract method calculateArea()
   - A concrete method display() that prints "Calculating area..."
2. Create two child classes: Rectangle and Circle
3. Each child must implement the calculateArea() method

Polymorphism vs Abstraction

Polymorphism:

- Multiple forms of same entity
- Focus on flexibility
- One interface, many implementations
- Example: Coffee machine button

Abstraction:

- Hiding implementation details
- Focus on simplicity
- Show what, hide how
- Example: Car dashboard

When to Use What?

Use Polymorphism when:

- You need different behaviors for the same operation
- You want to write flexible, extensible code
- You have parent-child relationships

Use Abstraction when:

- You want to hide complex implementation
- You want to define a contract for classes
- You need to focus on essential features only

What is a Contract?

A **contract** is a formal agreement that defines:-

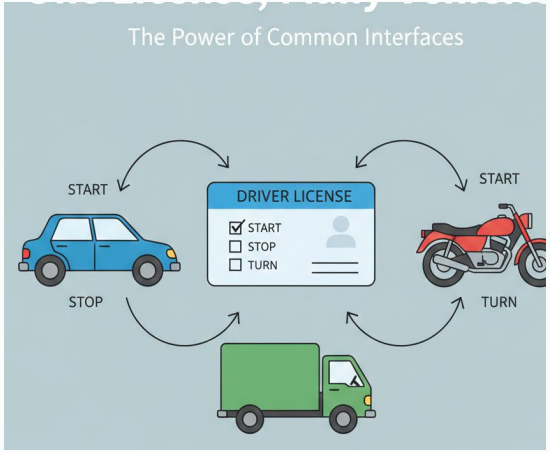
- What methods a class MUST have
- What parameters those methods MUST accept
- What those methods MUST return

Think of it like a **job description** - it tells you what you need to do, but not HOW to do it.

Real-Life Example:

Imagine a **Driver's License Contract**:

- You MUST know how to start the vehicle
- You MUST know how to stop the vehicle
- You MUST know how to turn



But: You can drive a car, bike, or truck - the METHOD is different, but the CONTRACT is the same!

In TypeScript, we create contracts using:

1. **Interfaces** (Most common)
2. **Abstract Classes** (When you need some shared code)

Summary

Polymorphism:

- One name, many forms
- Achieved through overloading and overriding
- Makes code flexible and reusable

Abstraction:

- Hide complexity, show simplicity
- Achieved through abstract classes and interfaces
- Makes code secure and maintainable

Together they make OOP powerful!

Newton School of Technology

Thank You!